CUET PG 2026/2027 | MCA & M.Sc. CS | Hinglish Edition | Classes 01 - 13 Complete

CUET PG Computer Science & MCA Data Structures Complete Master Notes

Complete 13-Lecture Syllabus | Easy Hinglish Explanations | Verified Theorems | Error Corrections | Quick Revision Sheet

Key Features & Course Overview

- Complete Class Coverage: Introduction, Arrays, Stacks, Queues, Linked Lists, Trees, BST, AVL, B/B+ Trees, Hashing, Graphs, Sorting & Searching.
- Easy Hinglish Explanations: Concepts explained in simple, beginner-friendly Hindi-English with real-world analogies.
- Mathematical Rigor: Derivations of Row/Column Major addresses, Tree theorems (n0 = n2 + 1), AVL height formulas, and Handshaking Lemma.
- Audited & Corrected: Identification and correction of technical errors found in the original lecture slides and daily class notes.
- CUET PG Exam Focus: High-priority complexity tables, Catalan numbers, common exam traps, and rapid revision cheat sheet.

Important Preparation Directive:

Data Structures and Algorithms carry approximately 12 to 14 direct questions (out of 75 domain questions) in the CUET PG Computer Science / MCA paper. Focus heavily on time complexities, pointer manipulation, tree traversals, and formula applications.

Table of Contents - Complete Course Syllabus

Yeh master notes CUET PG Computer Science / MCA ke complete 13 lectures ko cover karta hai. Har unit mein conceptual explanations, diagrams, solved examples, aur exam-oriented tips diye gaye hain:

Unit 1: Introduction to Data Structures & Algorithm Analysis
Reference: Class 01: Introduction to Data Structure | Data structure definition, primitive vs non-primitive, linear vs non-linear, static vs dynamic, core operations, and asymptotic complexity analysis.

Unit 2: Arrays & Memory Address Calculation
Reference: Class 02: Array data structure | 1D Arrays, 2D Arrays, Contiguous memory, Row-Major Order (RMO), Column-Major Order (CMO), Address calculation formulas with step-by-step solved numericals.

Unit 3: Stack Data Structure & Expressions
Reference: Class 03: Stack data structure | LIFO principle, Push/Pop operations, Overflow/Underflow, Polish Notation, Infix to Postfix conversion with stack, Postfix evaluation dry run.

Unit 4: Queues & Linked Lists
Reference: Class 04: Queue and Linked List Data Structure | FIFO principle, Linear Queue limitation, Circular Queue modulo formulas, Deque, Priority Queue, Singly/Doubly/Circular Linked Lists, Pointer manipulations.

Unit 5: Trees & Binary Tree Properties
Reference: Class 05: Binary tree | Tree terminology, Full, Complete, Perfect, Skewed binary trees, Mathematical theorems (n0 = n2 + 1), Catalan number, Inorder/Preorder/Postorder traversals.

Unit 6: Binary Search Tree (BST) & AVL Trees
Reference: Class 06: Binary Search Tree & Class 07: AVL Tree | BST property, Inorder sorted theorem, BST Search/Insert/Delete cases, AVL Tree self-balancing, Balance Factor, 4 Rotations (LL, RR, LR, RL), Minimum nodes formula.

Unit 7: B-Trees & B+ Trees
Reference: Class 08: B Tree, B+ Tree | Multiway search trees, Order m properties, Min/Max keys and children, B-Tree vs B+ Tree structural differences, Database index application.

Unit 8: Hashing & Collision Resolution
Reference: Class 09: Hashing | Hash table, Hash functions, Collision resolution: Linear Probing, Quadratic Probing, Double Hashing, Separate Chaining, Load Factor alpha, Complexity analysis.

Unit 9: Graph Theory & Graph Traversals
Reference: Class 10: Graph Theory | Graph terminology, Handshaking Lemma, Complete/Bipartite/Planar graphs, Adjacency Matrix vs List, Breadth-First Search (BFS), Depth-First Search (DFS).

Unit 10: Searching & Sorting Algorithms
Reference: Class 11 & Class 12: Searching and Sorting Algorithms | Linear vs Binary Search, Bubble, Selection, Insertion, Merge, Quick, Heap, Radix sorts. Stability, in-place behavior, divide & conquer, and master comparison matrix.

Unit 11: Critical Audit & Corrections of Original Class Slides
Reference: Quality Assurance & Error Verification | Comprehensive list of technical inaccuracies and ambiguities identified in the original PW Samarth lecture slides and notes, with verified corrections for CUET PG.

Unit 12: Master Comparison Tables
Reference: Cross-Topic Synthesis | Comprehensive head-to-head comparisons of frequently confused data structures and algorithms.

Unit 13: CUET PG Rapid Revision Cheat Sheet
Reference: Final Exam Preparation & Formula Digest | High-priority formulas, asymptotic rankings, recurrence relations, Catalan numbers, and last-minute exam pointers.

[INFO] How to Use These Notes for Maximum Score in CUET PG

- Har topic ko pehle easy Hinglish explanation se samjhein.
- Solved numericals (jaise 2D array address, AVL minimum nodes, Handshaking lemma) ko rough notebook mein khud solve karein.
- Exam se pehle "Unit 13: Rapid Revision Cheat Sheet" aur har unit ke "Must Remember" boxes ko revise karein.
- In notes ke saath humare second PDF "CUET_PG_Data_Structures_100_Plus_MCQ_Bank.pdf" ke saare 100+ questions attempt karein.

Unit 1: Introduction to Data Structures & Algorithm Analysis

Class Reference: Class 01: Introduction to Data Structure

1.1 Data Structure Kya Hai? (Conceptual Foundation)

Data Structure ek organized aur systematic tarika hai computer memory ke andar data ko store aur arrange karne ka, taaki hum us data par efficiently operations jaise access, search, insert, delete aur sort perform kar sakein.

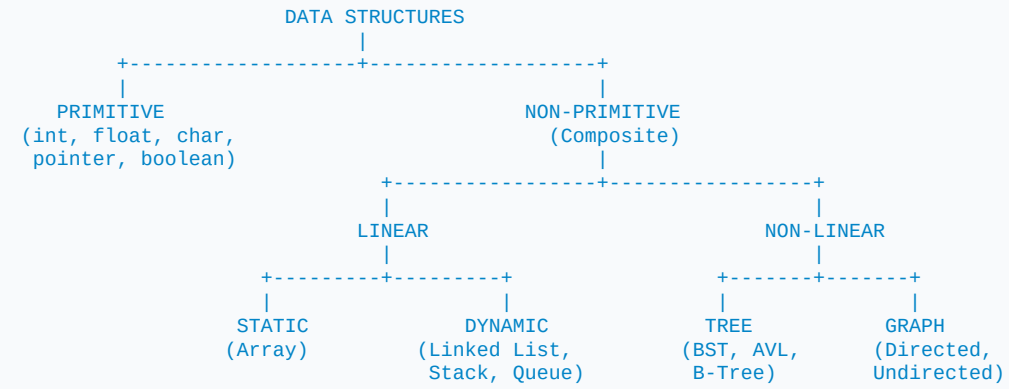
Simple shabdon mein: Agar hamare paas bohot saara data hai aur hum usko bina kisi structure ke rakh denge, toh search karne mein bohot zyada time lagega. Lekin agar data structured hai (jaise phone gallery mein albums ya dictionary mein sorted words), toh search karna instant ho jata hai.

- **Data Item:** Ek single unit of value (jaise Roll Number: 101, Name: "Aman").
- **Entity & Attribute:** Entity ek real-world object hota hai (jaise Student) aur uske properties ko attributes kehte hain (jaise Age, Marks).
- **Abstract Data Type (ADT):** ADT ek mathematical model hai jo yeh define karta hai ki data par KYA operations perform honge, lekin yeh NAHI batata ki unhe KAISE implement kiya jayega. Example: Stack ADT kehta hai PUSH aur POP hoga, chahe array se banao ya linked list se.

1.2 Classification of Data Structures

Data Structures ko mainly 3 alag-alag criteria ke basis par classify kiya jata hai:

[DIAGRAM & VISUAL TRACE] Complete Classification Tree of Data Structures



Primitive types computer hardware dwara directly supported hote hain. Non-primitive types primitive types ko combine karke banaye jaate hain.

A. Primitive vs Non-Primitive Data Structures

1. Primitive Data Structures: Yeh programming language ke basic built-in data types hote hain jo single value store karte hain aur machine level par directly operate hote hain. Examples: int, float, char, pointer, boolean.
2. Non-Primitive Data Structures: Yeh primitive data types ko combine karke banaye gaye complex structures hote hain jo multiple values ko group karte hain. Examples: Array, Linked List, Stack, Queue, Tree, Graph.

B. Linear vs Non-Linear Data Structures

1. Linear Data Structures: Isme elements ek sequential (one-after-another) order mein arrange hote hain. Har element ka ek unique predecessor (previous) aur successor (next) hota hai (first aur last ko chhodkar). Example: Array, Linked List, Stack, Queue. Single level memory traversal hoti hai.
2. Non-Linear Data Structures: Isme data elements sequential order mein nahi hote, balki hierarchical ya interconnected network relationship mein judte hain. Ek element ke multiple next ya previous elements ho sakte hain. Example: Trees (hierarchical) aur Graphs (network).

C. Static vs Dynamic Data Structures

1. Static Data Structure: Memory size compile-time par fix ho jata hai aur runtime par grow ya shrink nahi ho sakta. Elements contiguous memory mein store hote hain. Example: Array.
2. Dynamic Data Structure: Memory runtime par allocate aur deallocate hoti hai (using malloc/free ya new/delete). Size zarurat ke mutabiq grow aur shrink ho sakta hai. Elements non-contiguous memory mein pointers ke through linked hote hain. Example: Linked List, Dynamic Trees.

1.3 Operations on Data Structures

Har data structure par 6 fundamental operations kiye jaate hain:

- **Traversing:** Data structure ke har ek element ko exactly ek baar visit karna taaki usko process/print kiya ja sake.
- **Searching:** Diye gaye key value ka location (index ya memory address) find karna.
- **Inserting:** Structure ke andar naya element add karna.
- **Deleting:** Structure se kisi existing element ko remove karna.
- **Sorting:** Elements ko kisi specific logical order (Ascending ya Descending) mein arrange karna.
- **Merging:** Do sorted lists ko combine karke ek single sorted list banana.

1.4 Algorithm Analysis & Asymptotic Notations

Kisi algorithm ki performance ko measure karne ke liye do factors dekhe jaate hain: Time Complexity (kitna time/steps leta hai) aur Space Complexity (kitni extra memory leta hai).

Asymptotic Notations function ke growth rate ko express karte hain jab input size n infinity ki taraf badhta hai:

Notation	Name	Mathematical Meaning	Exam Significance
Big-O (O)	Upper Bound	f(n) <= c * g(n) for all n >= n0	Worst-case performance ko represent karta hai
Big-Omega (Omega)	Lower Bound	f(n) >= c * g(n) for all n >= n0	Best-case performance ko represent karta hai
Big-Theta (Theta)	Tight Bound	c1*g(n) <= f(n) <= c2*g(n)	Average ya exact growth rate batata hai.
Little-o (o)	Strict Upper Bound	f(n) < c * g(n) asymptotically	f(n) strictly chhota hai, equal nahi ho sakta.
Little-omega (<'•	Strict Lower Bound	f(n) > c * g(n) asymptotically	f(n) strictly bada hai.

[MOST IMPORTANT FOR CUET PG] Asymptotic Growth Rate Order (CUET PG Must-Memorize!)

O(1) < O(log log n) < O(log n) < O(sqrt(n)) < O(n) < O(n log n) < O(n^2) < O(n^3) < O(2^n) < O(n!) < O(n^n)
Tip: Exam mein options ko compare karne ke liye n = 16 ya n = 64 rakh kar check karein!

Exam-Focused Review: Unit 1

[MOST IMPORTANT FOR CUET PG] Most Important for CUET PG

- Linear vs Non-linear distinction: Array, Stack, Queue, Linked List are LINEAR; Tree and Graph are NON-LINEAR.
- Static vs Dynamic: Array size is fixed at compile-time; Linked list grows/shrinks dynamically at runtime.
- Asymptotic complexity hierarchy: Logarithmic O(log n) is strictly faster than Polynomial O(n^k) and Exponential O(2^n).

[MUST REMEMBER] Must Remember Rules & Formulas

- Definition of ADT: ADT specifies WHAT operations to do, NOT HOW to implement them.
- Handshaking degree property starts from graph concepts introduced in Unit 1.
- Pointers store memory addresses, not the data values themselves.

[COMMON EXAM TRAP / CORRECTION] Common Confusions & Exam Traps

- Trap: "Linked List is a non-linear data structure because it uses pointers." -> FALSE! Linked list is LINEAR because elements are logically sequential.
- Trap: "Array access is O(n)." -> FALSE! Array access using index is O(1) random access.
- Trap: Big-O is not ALWAYS worst case by definition (it is an upper bound), but in standard exam questions it is used to denote worst case.

Unit 2: Arrays & Memory Address Calculation

Class Reference: Class 02: Array data structure

2.1 Array Fundamentals & Memory Representation

Array ek homogeneous (same data type) elements ka fixed-size collection hai jo contiguous (lagaatar) memory locations mein store hota hai.

Contiguous storage ki wajah se Array direct index-based calculation se kisi bhi element ko constant time O(1) mein access kar sakta hai (Random Access).

[DIAGRAM & VISUAL TRACE] 1-Dimensional Array Memory Mapping

```
Index:       0       1       2       3       4
Value:     [ 10  |  20  |  30  |  40  |  50  ]
Address:   1000    1004    1008    1012    1016   (Assuming 4 bytes per element)
Base Address (B) = 1000, Element Size (W) = 4 bytes
```

Har agle element ka address pichhle element se exact W bytes aage hota hai.

2.2 Address Calculation in 1D Array

Agar array ka lower bound LB ho, Base Address B ho aur har element ka size W bytes ho, toh index i ka address hoga:

[MUST REMEMBER] 1D Array Address Formula

Address(A[i]) = Base Address (B) + W * (i - Lower_Bound)
Agar indexing 0 se start hoti hai (LB = 0), toh: Address(A[i]) = B + W * i

2.3 2-Dimensional Array Address Calculation (Row-Major vs Column-Major)

2D array computer memory (jo ki 1D linear hoti hai) mein do tareeqon se map hota hai:

1. Row-Major Order (RMO): Pehle row 0 ke saare elements store hote hain, fir row 1 ke, fir row 2 ke. (C, C++, Java use karte hain).
2. Column-Major Order (CMO): Pehle column 0 ke saare elements store hote hain, fir col 1 ke, fir col 2 ke. (FORTRAN, MATLAB use karte hain).

[MOST IMPORTANT FOR CUET PG] Formulas for 2D Array A[LB1..UB1, LB2..UB2]

Rows (R) = UB1 - LB1 + 1
Columns (C) = UB2 - LB2 + 1
Element size = W bytes, Base Address = B

1. Row-Major Order (RMO):
   Address(A[i][j]) = B + W * [(i - LB1) * C + (j - LB2)]

2. Column-Major Order (CMO):
   Address(A[i][j]) = B + W * [(j - LB2) * R + (i - LB1)]

Step-by-Step Solved CUET PG Numerical Problem

Question: Ek array A[-5..15, 10..30] memory mein store hai with Base Address B = 2000 aur har element 4 bytes leta hai. Find the address of A[5][20] in:
(a) Row-Major Order (b) Column-Major Order.

Solution Step-by-Step:
• LB1 = -5, UB1 = 15 => Rows R = 15 - (-5) + 1 = 21
• LB2 = 10, UB2 = 30 => Columns C = 30 - 10 + 1 = 21
• Target: i = 5, j = 20. Element size W = 4, Base Address B = 2000.

(a) Row-Major Order (RMO):
Address = B + W * [(i - LB1) * C + (j - LB2)]

= 2000 + 4 * [(5 - (-5)) * 21 + (20 - 10)]
= 2000 + 4 * [10 * 21 + 10]
= 2000 + 4 * [210 + 10] = 2000 + 4 * 220 = 2000 + 880 = 2880.

(b) Column-Major Order (CMO):
Address = B + W * [(j - LB2) * R + (i - LB1)]
= 2000 + 4 * [(20 - 10) * 21 + (5 - (-5))]
= 2000 + 4 * [10 * 21 + 10]
= 2000 + 4 * [210 + 10] = 2000 + 4 * 220 = 2880.

2.4 Array Operations Time Complexities

Operation	Best Case	Average Case	Worst Case	Reason / Working
Access by Index	O(1)	O(1)	O(1)	Direct base address offset calculation
Search (Linear)	O(1)	O(n)	O(n)	Element at first position vs end/not found
Search (Binary - Sorted)	O(1)	O(log n)	O(log n)	Requires sorted array, divides in half
Insertion at End	O(1)	O(1)	O(1)	If array has free space at end
Insertion at Beginning	O(n)	O(n)	O(n)	All n elements must be shifted right
Deletion at Beginning	O(n)	O(n)	O(n)	All elements must be shifted left
Deletion at End	O(1)	O(1)	O(1)	Just decrement size variable

Exam-Focused Review: Unit 2

[MOST IMPORTANT FOR CUET PG] Most Important for CUET PG

- Row-Major vs Column-Major calculation is guaranteed to appear in CUET PG/MCA.
- Remember: In RMO, multiply row offset by number of COLUMNS (C). In CMO, multiply column offset by number of ROWS (R).
- Array insert/delete in worst case is O(n) because of shifting, but access is O(1).

[MUST REMEMBER] Must Remember Rules & Formulas

- Number of elements in A[LB..UB] = UB - LB + 1.
- Number of elements in 2D array = (UB1 - LB1 + 1) * (UB2 - LB2 + 1).
- Zero-based indexing has LB = 0; 1-based indexing has LB = 1.

[COMMON EXAM TRAP / CORRECTION] Common Confusions & Exam Traps

- Trap: Forgetting to add 1 when calculating number of rows/columns: UB - LB is WRONG; UB - LB + 1 is CORRECT.
- Trap: Using Rows instead of Columns in Row-Major formula. RMO walks across columns!

Unit 3: Stack Data Structure & Expressions

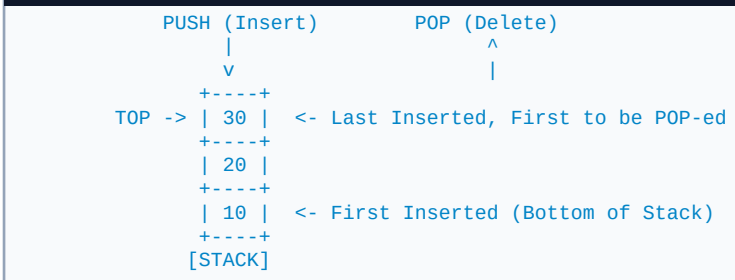
Class Reference: Class 03: Stack data structure

3.1 Stack Definition & Core Principles

Stack ek linear data structure hai jisme insertion aur deletion sirf ek hi end se hota hai, jisko TOP kehte hain.

Yeh LIFO (Last In First Out) ya FILO (First In Last Out) principle follow karta hai. Matlab jo element sabse aakhir mein insert hoga, woh sabse pehle bahar niklega.

[DIAGRAM & VISUAL TRACE] Stack PUSH and POP Operations (LIFO)



TOP pointer always points to the top-most valid element in the stack.

Boundary Conditions

1. Stack Overflow: Jab stack full ho (Array implementation mein TOP == MAX - 1) aur naya element PUSH karne ki koshish ki jaye.
2. Stack Underflow: Jab stack khali ho (TOP == -1) aur POP karne ki koshish ki jaye.

3.2 Arithmetic Expressions & Polish Notation

Expressions 3 alag notations mein likhi ja sakti hain:
• Infix: Operator operands ke beech mein hota hai: A + B
• Prefix (Polish Notation): Operator operands se pehle hota hai: + A B
• Postfix (Reverse Polish Notation): Operator operands ke baad hota hai: A B +

[INFO] Operator Precedence and Associativity Table

1. Parentheses: ( ) -> Highest Priority
2. Exponentiation: ^ -> Priority 3, Associativity: RIGHT to LEFT (e.g., 2^3^2 = 2^(3^2) = 2^9 = 512)
3. Multiplication / Division: *, / -> Priority 2, Associativity: LEFT to RIGHT
4. Addition / Subtraction: +, - -> Priority 1, Associativity: LEFT to RIGHT

3.3 Infix to Postfix Conversion Algorithm (Stack Trace)

Infix ko Postfix mein convert karne ke liye Stack operator storage ke roop mein use hota hai:
• Operands ko directly output mein likha jata hai.
• Left parenthesis "(" ko stack mein push karte hain.
• Right parenthesis ")" aane par, stack se tab tak pop karte hain jab tak "(" na mile.
• Operator aane par: Stack ke top par agar higher ya equal precedence ka operator ho toh use pop karke output mein bhejte hain, fir current operator ko push karte hain (Except for ^ operator which is Right-to-Left).

[DIAGRAM & VISUAL TRACE] Complete Dry Run: Convert Infix "A + B * (C ^ D - E)" to Postfix

Token	Action / Rule	Operator Stack	Postfix Output
A	Operand -> Append to output	Empty	A
+	Operator -> Push to stack	+	A
B	Operand -> Append to output	+	A B
*	'*' has higher priority than '+'	+ *	A B
(	Left parenthesis -> Push to stack	+ * (	A B
C	Operand -> Append to output	+ * (	A B C
^	Operator -> Push to stack	+ * (^	A B C
D	Operand -> Append to output	+ * (^	A B C D
-	'-' has lower priority than '^'	+ * (-	A B C D ^
E	Operand -> Append to output	+ * (-	A B C D ^ E
)	Pop till '('	+ *	A B C D ^ E -
End	Pop all remaining operators	Empty	A B C D ^ E - * +

Final Postfix Output: A B C D ^ E - * +

3.4 Evaluation of Postfix Expression using Stack

Algorithm:
1. Expression ko left to right scan karo.
2. Agar operand mile, toh use Stack mein PUSH karo.
3. Agar operator mile, toh stack se 2 operands POP karo: op2 = POP(), op1 = POP().
4. Calculate result = op1 (operator) op2, aur result ko firse Stack mein PUSH karo.

5. End mein stack ka single element hi final answer hota hai.

[COMMON EXAM TRAP / CORRECTION] Order of Operands in Evaluation (Exam Trap!)

Jab operator minus (-) ya divide (/) ho, toh sequence dhyan rakhein:
op2 = POP() (pehle pop hua)
op1 = POP() (doosre number pe pop hua)
Result = op1 - op2 ya op1 / op2 (NOT op2 - op1!)

Exam-Focused Review: Unit 3

[MOST IMPORTANT FOR CUET PG] Most Important for CUET PG

- Postfix evaluation order: op1 is second popped, op2 is first popped -> result = op1 - op2.
- Exponentiation operator (^) associativity is RIGHT to LEFT.
- Applications of Stack: Function call execution stack, Recursion, Undo/Redo operations, Expression evaluation, Syntax parsing (balanced parentheses).

[MUST REMEMBER] Must Remember Rules & Formulas

- Stack Overflow condition: TOP == MAX - 1 (Array implementation).
- Stack Underflow condition: TOP == -1.
- Minimum stacks needed to implement a Queue = 2 Stacks.

[COMMON EXAM TRAP / CORRECTION] Common Confusions & Exam Traps

- Trap: In Infix to Postfix, parentheses are NEVER included in the final output string.
- Trap: Assuming 2^3^2 = (2^3)^2 = 64. WRONG! Exponentiation is right-associative: 2^(3^2) = 2^9 = 512.

Unit 4: Queues & Linked Lists

Class Reference: Class 04: Queue and Linked List Data Structure

4.1 Queue Fundamentals & Operations

Queue ek linear data structure hai jo FIFO (First In First Out) principle follow karta hai.
• Insertion REAR end par hota hai (ENQUEUE operation).
• Deletion FRONT end se hota hai (DEQUEUE operation).
• Real-life example: Ticket counter line, CPU task scheduling, Printer spooler.

[DIAGRAM & VISUAL TRACE] Queue Operations (FIFO)



Element enter hota hai Rear se, aur leave karta hai Front se.

4.2 Linear Queue vs Circular Queue

Linear Queue ki sabse badi kami hoti hai "False Overflow": Agar elements delete ho chuke hain aur front aage badh chuka hai, tab bhi rear == MAX - 1 hone par queue naye elements insert nahi kar pati, bhale hi aage ki memory khali ho.

Is problem ko solve karta hai Circular Queue, jisme array ke aakhiri slot ke baad pehla slot connect ho jata hai using modulo arithmetic.

[MOST IMPORTANT FOR CUET PG] Circular Queue Formulas (N = Queue Capacity)

1. Circular Increment:
   REAR = (REAR + 1) % N
   FRONT = (FRONT + 1) % N

2. Full Condition (Overflow):
   (REAR + 1) % N == FRONT

3. Empty Condition (Underflow):
   FRONT == -1 (when initialized with -1)
   Or FRONT == REAR (depending on design)

4. Count of elements in Circular Queue:
   Count = (REAR - FRONT + N) % N

4.3 Types of Queues

- **Linear Queue:** Simple FIFO structure, suffers from memory wastage (false overflow).
- **Circular Queue:** Last element points to first element; utilizes memory efficiently using modulo N.
- **Deque (Double-Ended Queue):** Insertion aur deletion dono ends (Front aur Rear) se allow hota hai. Input-restricted deque (insert at one end, delete at both) aur Output-restricted deque (delete at one end, insert at both).
- **Priority Queue:** Har element ke saath ek priority associate hoti hai. Highest priority element pehle process hota hai. Internally implemented using Binary Heap.

4.4 Linked List Data Structure

Linked List ek linear dynamic data structure hai jisme elements (nodes) non-contiguous memory locations par store hote hain aur pointers ke through ek doosre se judte hain.

Har Node ke 2 parts hote hain:
1. DATA: Actual value jo store karni hai.
2. NEXT: Agle node ka memory address (pointer).

[DIAGRAM & VISUAL TRACE] Singly, Doubly, and Circular Linked Lists

```
1. Singly Linked List (SLL):
   HEAD -> [ Data | Next ] -> [ Data | Next ] -> [ Data | NULL ]

2. Doubly Linked List (DLL):
   NULL <- [ Prev | Data | Next ] <==> [ Prev | Data | Next ] -> NULL

3. Circular Linked List (CLL):
   HEAD -> [ Data | Next ] -> [ Data | Next ] --+
             ^                                  |
             +----------------------------------+
```

SLL can traverse only forward. DLL can traverse both forward and backward. CLL has no NULL pointer.

Operation	Array	Singly Linked List	Doubly Linked List
Access k-th element	O(1) [Random Access]	O(k) [Sequential]	O(k) [Sequential]
Insert at Beginning	O(n) [Needs shifting]	O(1) [Change HEAD pointer]	O(1) [Change pointers]
Insert at End	O(1) [If space available]	O(n) or O(1) [with TAIL]	O(1) [with TAIL pointer]
Delete at Beginning	O(n) [Needs shifting]	O(1) [HEAD = HEAD->next]	O(1)
Delete at End	O(1)	O(n) [Need prev pointer]	O(1) [with TAIL pointer]
Memory Overhead	No pointer overhead	1 pointer per node	2 pointers per node

Exam-Focused Review: Unit 4

[MOST IMPORTANT FOR CUET PG] Most Important for CUET PG

- Circular Queue Full condition: (rear + 1) % N == front.
- Minimum Queues needed to implement a Stack = 2 Queues.
- Linked List reversal requires 3 pointers: prev, current, next.

[MUST REMEMBER] Must Remember Rules & Formulas

- In Singly Linked List, deleting the last node takes O(n) even if TAIL pointer is given, because we must update the second-to-last node next to NULL.
- In Doubly Linked List, deleting the last node with TAIL pointer takes O(1).

[COMMON EXAM TRAP / CORRECTION] Common Confusions & Exam Traps

- Trap: "Queue insert and delete both happen at the front." -> WRONG! Insert at REAR, Delete at FRONT.
- Trap: Linear queue overflow check (rear == MAX - 1) does not mean the queue is actually full if front > 0.

Unit 5: Trees & Binary Tree Properties

Class Reference: Class 05: Binary tree

5.1 Tree Terminology & Concepts

Tree ek non-linear, hierarchical data structure hai jo nodes aur edges se milkar banta hai. Isme ek single ROOT node hota hai aur koi cycles (loops) nahi hote.

Tree ke do nodes ke beech hamesha exactly EK unique path hota hai. Agar tree mein n nodes hain, toh exactly n - 1 edges hongi.

- **Root:** Tree ka top-most node jiska koi parent nahi hota.
- **Leaf Node (Terminal):** Aise nodes jinke 0 children hote hain (Degree = 0).
- **Internal Node (Non-Terminal):** Aise nodes jinke paas kam se kam 1 child ho.
- **Degree of a Node:** Us node ke total children ki sankhya.
- **Degree of a Tree:** Tree ke sabhi nodes mein se maximum degree.
- **Level:** Root level 0 par hota hai, root ke children level 1 par, and so on.
- **Height of Node:** Us node se kisi bhi leaf node tak ka longest path (number of edges).
- **Depth of Node:** Root se us node tak ka path length.

5.2 Types of Binary Trees

Binary Tree ek aisi tree hai jisme har node ke AT MOST 2 children ho sakte hain (Left Child aur Right Child).

Type of Binary Tree	Formal Definition	CUET PG Identifying Property
Full / Strict Binary Tree	Har node ke ya toh 0 ya 2 children hote hain. Kisi bhi node ka 1 child nahi hota.	No node has degree 1.
Complete Binary Tree	Saare levels completely filled hote hain except possibly the last level, jo left to right fill hota hai.	Used to represent Binary Heap in an Array.
Perfect Binary Tree	Saare internal nodes ke 2 children hote hain aur saare leaf nodes same level par hote hain.	Total nodes for height h = 2^(h+1) - 1.
Degenerate / Skewed Tree	Har internal node ka sirf 1 child hota hai. Yeh basically ek Linked List ban jata hai.	Height = n - 1, Search time = O(n).

5.3 Crucial Mathematical Theorems (Guaranteed CUET PG Questions!)

[MOST IMPORTANT FOR CUET PG] Theorem 1: Leaf Nodes vs Degree-2 Nodes (n0 = n2 + 1)

Kisi bhi non-empty Binary Tree mein:
Number of Leaf Nodes (n0) = Number of Nodes with Degree 2 (n2) + 1
Formula: n0 = n2 + 1

Proof / Logic:
Total nodes n = n0 + n1 + n2 (where n1 = nodes with 1 child)
Total edges e = n - 1
Also, every edge comes from a parent: e = 0*n0 + 1*n1 + 2*n2 = n1 + 2*n2
Equating both: n - 1 = n1 + 2*n2
(n0 + n1 + n2) - 1 = n1 + 2*n2
n0 - 1 = n2 ==> n0 = n2 + 1 [PROVED!]

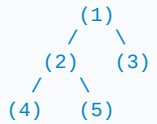
Other Essential Binary Tree Formulas

1. Level i par maximum nodes: 2^i (assuming root at level 0).
2. Height h waale Binary Tree mein maximum nodes: N_max = 2^(h+1) - 1.
3. Height h waale Binary Tree mein minimum nodes: N_min = h + 1.
4. n nodes waale Binary Tree ki minimum height: h_min = ceil(log2(n + 1)) - 1 = O(log n).
5. n nodes waale Binary Tree ki maximum height: h_max = n - 1 = O(n) (Skewed tree).
6. n unlabelled nodes se kitne distinct binary trees ban sakte hain? Catalan Number:
   C_n = (1 / (n + 1)) * (2n choose n) = (2n)! / ((n + 1)! * n!)

5.4 Binary Tree Traversals

Binary Tree ke nodes ko visit karne ke 4 main tareeqe hote hain:

[DIAGRAM & VISUAL TRACE] Tree Traversal Example



- Inorder (Left, Root, Right): 4 -> 2 -> 5 -> 1 -> 3
- Preorder (Root, Left, Right): 1 -> 2 -> 4 -> 5 -> 3
- Postorder (Left, Right, Root): 4 -> 5 -> 2 -> 3 -> 1
- Level Order: 1 -> 2 -> 3 -> 4 -> 5

Unique tree reconstruction ke liye INORDER traversal ke saath PREORDER ya POSTORDER ka hona mandatory hai.

- **Inorder (LNR):** Left Subtree -> Node (Root) -> Right Subtree
- **Preorder (NLR):** Node (Root) -> Left Subtree -> Right Subtree
- **Postorder (LRN):** Left Subtree -> Right Subtree -> Node (Root)
- **Level-Order:** Level by level, left to right using a FIFO Queue.

Exam-Focused Review: Unit 5

[MOST IMPORTANT FOR CUET PG] Most Important for CUET PG

- Always remember n0 = n2 + 1 for ANY binary tree (not just full binary tree).
- Inorder + Preorder OR Inorder + Postorder is required to construct a unique binary tree. Preorder + Postorder CANNOT construct a unique binary tree!
- Catalan number formula gives number of structurally distinct binary trees with n nodes.

[MUST REMEMBER] Must Remember Rules & Formulas

- For n = 3 nodes: Catalan C3 = (1/4) * (6! / (3! * 3!)) = 5 distinct binary trees.
- Maximum nodes in a binary tree of height 4 = 2^(4+1) - 1 = 31.

[COMMON EXAM TRAP / CORRECTION] Common Confusions & Exam Traps

- Trap: Confusing tree height definition: some books count edges (root height 0), some count nodes (root height 1). CUET PG standard follows edge count (height = levels - 1).

Unit 6: Binary Search Tree (BST) & AVL Trees

Class Reference: Class 06: Binary Search Tree & Class 07: AVL Tree

6.1 Binary Search Tree (BST) Properties

Binary Search Tree ek binary tree hai jisme har node par yeh property hold karti hai:
• Left Subtree ke sabhi keys < Node ki Key.
• Right Subtree ke sabhi keys > Node ki Key.
• Dono subtrees bhi recursively BST hone chahiye.

GOLDEN THEOREM: Kisi bhi BST ka Inorder Traversal hamesha STRICTLY ASCENDING (Sorted) order produce karta hai!

[MOST IMPORTANT FOR CUET PG] BST Operations Time Complexities

```
Operation  | Average Case (Balanced) | Worst Case (Skewed)
-----------+-------------------------+--------------------
Search     | O(log n)                | O(n)
Insertion  | O(log n)                | O(n)
Deletion   | O(log n)                | O(n)
Finding Min| O(log n)                | O(n) [Leftmost node]
Finding Max| O(log n)                | O(n) [Rightmost node]
```

Worst case tab hota hai jab input already sorted ho (e.g., 10, 20, 30, 40), jisse skewed tree banta hai.

6.2 BST Deletion Cases

- **Case 1: Node is a Leaf (0 Children):** Directly delete the node and set parent pointer to NULL. Complexity: O(1) after finding.
- **Case 2: Node has 1 Child:** Node ko bypass karke uske parent ko directly child se connect kar do. Complexity: O(1).
- **Case 3: Node has 2 Children:** Node ko replace karo ya toh uske Inorder Predecessor se (left subtree ka maximum) ya Inorder Successor se (right subtree ka minimum), fir us predecessor/successor node ko delete karo.

6.3 AVL Tree (Height-Balanced BST)

AVL Tree (named after Adelson-Velsky and Landis) ek self-balancing Binary Search Tree hai jisme har node par Balance Factor (BF) -1, 0, ya +1 hota hai.

Balance Factor (BF) Formula:
BF(node) = Height(Left Subtree) - Height(Right Subtree)

Agar kisi insertion ya deletion ke baad kisi node ka BF -1, 0, +1 ke bahar chala jaye (jaise +2 ya -2), toh tree unbalanced ho jata hai aur hum ROTATIONS perform karke balance restore karte hain.

Imbalance Case	Cause / Location of Insertion	Required Rotation	Nature of Rotation
LL Imbalance	Inserted into Left subtree of Left child	Single Right Rotation	Single rotation around node A
RR Imbalance	Inserted into Right subtree of Right child	Single Left Rotation	Single rotation around node A
LR Imbalance	Inserted into Right subtree of Left child	Double Rotation: Left then Right	Left rotate on left child, then Right rotate on node A
RL Imbalance	Inserted into Left subtree of Right child	Double Rotation: Right then Left	Right rotate on right child, then Left rotate on node A

[MUST REMEMBER] Minimum Nodes in AVL Tree of Height h (CUET PG Formula)

Recurrence Relation: N(h) = N(h - 1) + N(h - 2) + 1
Base Cases:
N(0) = 1 (Height 0: only root node)
N(1) = 2 (Height 1: root + 1 child)
N(2) = N(1) + N(0) + 1 = 2 + 1 + 1 = 4
N(3) = N(2) + N(1) + 1 = 4 + 2 + 1 = 7
N(4) = N(3) + N(2) + 1 = 7 + 4 + 1 = 12
Maximum Height for n nodes: h ~= 1.44 * log2(n) = O(log n).
Search, Insert, Delete in AVL Tree is ALWAYS O(log n) in Best, Average, and Worst Case!

Exam-Focused Review: Unit 6

[MOST IMPORTANT FOR CUET PG] Most Important for CUET PG

- Inorder traversal of BST gives keys in sorted order. Exam question: "Given preorder traversal, can we find the BST?" YES, because sorting preorder gives the Inorder traversal!
- AVL Tree ensures strict O(log n) worst-case time for search, insert, and delete.
- Number of rotations: LL and RR need 1 rotation; LR and RL need 2 rotations.

[MUST REMEMBER] Must Remember Rules & Formulas

- AVL Balance factor must be in {-1, 0, +1}.
- Minimum nodes formula N(h) = N(h-1) + N(h-2) + 1.

[COMMON EXAM TRAP / CORRECTION] Common Confusions & Exam Traps

- Trap: Confusing LR rotation: First rotation is LEFT on child, Second is RIGHT on parent. Students often reverse this!

Unit 7: B-Trees & B+ Trees

Class Reference: Class 08: B Tree, B+ Tree

7.1 B-Tree Fundamentals & Properties

B-Tree ek self-balancing, m-way search tree hai jo secondary storage devices (Hard Disks/SSDs) par large blocks of data ko efficiently read/write karne ke liye design kiya gaya hai.

Kyunki disk access bohot slow hota hai compared to RAM, B-Tree ek node ke andar multiple keys store karta hai, jisse tree ki height bohot kam rehti hai (Fat and Short tree).

[MOST IMPORTANT FOR CUET PG] B-Tree of Order m: Standard Properties (CUET PG Direct Questions!)

1. Maximum Children: Har node ke at most m children ho sakte hain.
2. Maximum Keys: Har node mein at most (m - 1) keys ho sakti hain.
3. Minimum Children (Non-root Internal Nodes): ceil(m / 2) children.
4. Minimum Keys (Non-root Internal Nodes): ceil(m / 2) - 1 keys.
5. Root Node Condition: Root ke kam se kam 2 children hone chahiye (agar leaf na ho) aur kam se kam 1 key honi chahiye.
6. All leaf nodes MUST be at the EXACT same level!

7.2 B-Tree vs B+ Tree Comparison

B+ Tree B-Tree ka an advanced variant hai jo relational databases (MySQL InnoDB, Oracle) mein table indexing ke liye use hota hai.

Feature	B-Tree	B+ Tree (Database Standard)
Data Storage Location	Data pointers internal nodes aur leaf nodes dono mein hote hain.	Data pointers ONLY leaf nodes mein hote hain. Internal nodes sirf search keys rakhte hain.
Leaf Node Linking	Leaf nodes aapas mein connected nahi hote.	Saare leaf nodes aapas mein Linked List ke through connected hote hain.
Range Queries	Range query slow hoti hai (tree traversal karna padta hai).	Range query extremely fast hoti hai (just traverse the leaf linked list).
Key Duplication	Keys duplicate nahi hoti (har key unique location par).	Search keys internal nodes mein duplicate hoti hain taaki leaf tak reach kar sakein.
Height of Tree	Thodi zyada height hoti hai kyunki internal nodes mein data bhi hota hai.	Bohot kam height hoti hai kyunki internal nodes zyada keys fit kar sakte hain.

Exam-Focused Review: Unit 7

[MOST IMPORTANT FOR CUET PG] Most Important for CUET PG

- Order m calculation: If m = 5, Max keys = 4, Min keys in non-root = ceil(5/2) - 1 = 3 - 1 = 2 keys.
- In B+ Tree, only leaf nodes store data records; leaves are linked as a linked list.
- All leaves in a B-Tree are at the same depth.

[MUST REMEMBER] Must Remember Rules & Formulas

- Root of a B-Tree has at least 2 children (if not leaf) and at least 1 key.
- Internal nodes have at least ceil(m/2) children.

[COMMON EXAM TRAP / CORRECTION] Common Confusions & Exam Traps

- Trap: Thinking B+ Tree is slower because keys are repeated. In reality, B+ Tree is FASTER for disk indexing because internal nodes store more keys per block, resulting in fewer disk I/O operations.

Unit 8: Hashing & Collision Resolution

Class Reference: Class 09: Hashing

8.1 Hashing Concept & Hash Functions

Hashing ek technique hai jisme hum key ko direct memory address (array index) mein map karte hain using a mathematical function called Hash Function: Index = h(key).

Ideal case mein Hashing O(1) constant time search, insertion, aur deletion provide karti hai.

- **Division Method:** h(k) = k mod m (jahan m table size hai, preferably a prime number).
- **Mid-Square Method:** k ka square karo, aur beech ke digits ko hash address ke roop mein lo.
- **Folding Method:** Key ko equal parts mein divide karo aur un parts ko add karke index nikalo.

- **Multiplication Method:** h(k) = floor(m * (k * A mod 1)), jahan 0 < A < 1 (Knuth suggests A ~= 0.618033).

8.2 Collision & Load Factor

Collision tab hota hai jab do alag-alag keys same hash index generate karti hain: h(k1) == h(k2) where k1 != k2.

Load Factor (alpha):
alpha = (Total number of elements n) / (Table size m)
• Separate chaining mein alpha > 1 ho sakta hai.
• Open addressing mein alpha <= 1 hona mandatory hai.

Collision Resolution Method	Probe Sequence Formula	Major Exam Characteristics
Linear Probing	h(k, i) = (h'(k) + i) mod m	Suffers from PRIMARY CLUSTERING (continuous blocks of occupied slots build up
Quadratic Probing	h(k, i) = (h'(k) + c1*i + c2*i^2) mod m	Suffers from SECONDARY CLUSTERING (keys hashing to same initial position follow
Double Hashing	h(k, i) = (h1(k) + i * h2(k)) mod m	Best open addressing method. Eliminates both primary and secondary clustering. h2(k) must
Separate Chaining (Open Hashing)	Each bucket maintains a Linked List	Never overflows. Search time = O(1 + alpha). Worst case = O(n) if all keys hash to same bucket.

Exam-Focused Review: Unit 8

[MOST IMPORTANT FOR CUET PG] Most Important for CUET PG

- Primary clustering is caused by Linear Probing; Secondary clustering by Quadratic Probing.
- Double Hashing uses two hash functions: h(k, i) = (h1(k) + i * h2(k)) % m.
- Average case search time in Hash Table is O(1); Worst case is O(n).

[MUST REMEMBER] Must Remember Rules & Formulas

- Load factor alpha = n / m.
- In Open Addressing, table size m MUST be greater than or equal to number of keys n.

[COMMON EXAM TRAP / CORRECTION] Common Confusions & Exam Traps

- Trap: Assuming worst-case search in Hash Table is always O(1). In worst case (all keys collide into same slot), it degrades to O(n)!

Unit 9: Graph Theory & Graph Traversals

Class Reference: Class 10: Graph Theory

9.1 Graph Terminology & Handshaking Theorem

Graph G = (V, E) ek non-linear data structure hai jo vertices V (nodes) aur edges E (connections) ka set hota hai.

[MOST IMPORTANT FOR CUET PG] Handshaking Lemma (Most Tested CUET PG Formula!)

Undirected Graph Theorem:
Sum of degrees of all vertices is equal to TWICE the number of edges!
Sum deg(v) = 2 * |E|

Corollary 1: Kisi bhi graph mein ODD DEGREE waale vertices ki sankhya hamesha EVEN hoti hai!
Corollary 2: Directed Graph mein:
Sum In-degree(v) = Sum Out-degree(v) = |E|

Special Graph Types

1. Complete Graph (Kn): Har vertex baaki sabhi vertices se directly connected hota hai. Total edges = n * (n - 1) / 2. Har vertex ki degree n - 1 hoti hai.
2. Bipartite Graph: Vertices ko do disjoint sets V1 aur V2 mein divide kiya ja sakta hai aise ki koi bhi edge same set ke do vertices ke beech na ho. Complete bipartite graph Km,n mein total edges = m * n.
3. Planar Graph: Aisa graph jisko plane par bina kisi intersecting edges ke draw kiya ja sake. Euler's Formula: V - E + F = 2 (where F is number of regions/faces).
4. Eulerian Graph: Graph contains an Eulerian circuit (visits every edge exactly once and returns to start) IFF every vertex has an EVEN degree.
5. Hamiltonian Graph: Graph contains a cycle that visits every VERTEX exactly once.

9.2 Graph Representation: Matrix vs List

Parameter	Adjacency Matrix	Adjacency List
Space Complexity	O(V^2) [Always takes V x V memory]	O(V + E) for undirected, O(V + E) for directed
Best For	Dense Graphs (|E| ~= V^2)	Sparse Graphs (|E| << V^2)
Check if edge (u, v) exists	O(1) [Direct lookup matrix[u][v]]	O(deg(u)) [Traverse list of u]
Find all neighbors of u	O(V) [Scan entire row u]	O(deg(u)) [Traverse only adjacent list]
Adding a Vertex	O(V^2) [Requires resizing matrix]	O(1) [Add new head node]

9.3 Graph Traversals: BFS vs DFS

Feature	Breadth-First Search (BFS)	Depth-First Search (DFS)
Underlying Data Structure	Queue (FIFO)	Stack (LIFO) or System Recursion
Traversal Strategy	Level by level (explores all immediate neighbors first)	Goes as deep as possible along each branch before backtracking
Time Complexity	O(V + E) using Adjacency List	O(V + E) using Adjacency List
Space Complexity	O(V) [Queue holds level nodes]	O(V) [Stack holds recursion path]
Shortest Path	Finds shortest path in unweighted graphs	Does NOT guarantee shortest path
Primary Applications	Shortest path, Web crawling, Social network friend distance	Cycle detection, Topological sort, Connected components, Maze solving

Exam-Focused Review: Unit 9

[MOST IMPORTANT FOR CUET PG] Most Important for CUET PG

- Handshaking lemma: Sum of degrees = 2 * E. Number of vertices with odd degree is always even!
- Maximum edges in simple undirected graph with n vertices = n(n-1)/2.
- BFS uses Queue; DFS uses Stack.

[MUST REMEMBER] Must Remember Rules & Formulas

- Adjacency Matrix space is O(V^2) regardless of edge count.
- Eulerian graph condition: Connected and all vertices have even degree.

[COMMON EXAM TRAP / CORRECTION] Common Confusions & Exam Traps

- Trap: Original slide error! Node 4 in the lecture slide had indegree 3, outdegree 3, total degree 6 (NOT outdegree 6).
- Trap: Forgetting that a tree with n vertices is a connected graph with exactly n - 1 edges and no cycles.

Unit 10: Searching & Sorting Algorithms

Class Reference: Class 11 & Class 12: Searching and Sorting Algorithms

10.1 Searching Algorithms

- **Linear Search:** Unsorted ya sorted array mein linearly element by element check karta hai. Best case: O(1), Worst case: O(n). Space: O(1).
- **Binary Search:** Array MUST be SORTED. Divide and conquer approach. Har step par search space half ho jata hai. Low, High, Mid = (Low + High) / 2. Time: O(log n), Space: O(1) iterative, O(log n) recursive.

10.2 Elementary Sorting Algorithms

- **Bubble Sort:** Adjacent elements compare hote hain aur swap hote hain agar galat order mein hon. Har pass ke baad largest element aakhir mein settle ho jata hai. Optimized with flag: Best Case O(n) (already sorted). Worst/Avg: O(n^2). Stable, In-Place.
- **Selection Sort:** Array ke unsorted part mein se minimum element dhoond kar use current position se swap karta hai. Hamesha n(n-1)/2 comparisons karta hai chahe array sorted ho ya nahi! Best/Avg/Worst: O(n^2). Number of swaps: Minimum O(n). UNSTABLE, In-Place.
- **Insertion Sort:** Card game ki tarah: Har naye element ko uske sahi sorted position par insert karta hai. Best Case O(n) (already sorted). Worst/Avg: O(n^2). STABLE, In-Place, ADAPTIVE. Sabse best algorithm for small arrays ya nearly-sorted data.

10.3 Advanced Sorting Algorithms

- **Merge Sort:** Divide and Conquer: Array ko recursively do halves mein divide karta hai jab tak single elements na bachein, fir sorted order mein merge karta hai. Recurrence: T(n) = 2T(n/2) + O(n). Best/Avg/Worst: O(n log n). STABLE, OUT-OF-PLACE (Extra O(n) space needed). Ideal for Linked Lists.
- **Quick Sort:** Divide and Conquer with Partitioning: Ek Pivot choose karta hai aur array ko partition karta hai (pivot se chhote left mein, bade right mein). Best/Avg: O(n log n). Worst Case: O(n^2) (jab input already sorted ho aur first/last element ko pivot banaya jaye). UNSTABLE, IN-PLACE (Auxiliary stack space O(log n)). Practical fastest in-memory sort.
- **Heap Sort:** Binary Heap banata hai: Max-Heap mein root hamesha largest hota hai. Root ko end se swap karke heap size ghatata hai aur heapify karta hai. Build Heap takes O(n), each extraction takes O(log n). Total time: O(n log n) in ALL cases. UNSTABLE, IN-PLACE (O(1) space).

10.4 Master Sorting Comparison Matrix (CUET PG Cheat Sheet)

Algorithm	Best Time	Average Time	Worst Time	Space	Stable?	In-Place?
Bubble Sort	O(n)	O(n^2)	O(n^2)	O(1)	YES	YES
Selection Sort	O(n^2)	O(n^2)	O(n^2)	O(1)	NO	YES
Insertion Sort	O(n)	O(n^2)	O(n^2)	O(1)	YES	YES
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	YES	NO
Quick Sort	O(n log n)	O(n log n)	O(n^2)	O(log n)	NO	YES
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1)	NO	YES
Counting Sort	O(n + k)	O(n + k)	O(n + k)	O(k)	YES	NO
Radix Sort	O(d*(n+k))	O(d*(n+k))	O(d*(n+k))	O(n+k)	YES	NO

Exam-Focused Review: Unit 10

[MOST IMPORTANT FOR CUET PG] Most Important for CUET PG

- Selection sort makes the minimum number of swaps: exactly O(n).
- Quick sort worst case O(n^2) happens when array is already sorted or reverse sorted and extreme element is chosen as pivot.
- Merge sort requires O(n) auxiliary memory, making it out-of-place for arrays.
- Heap sort has O(n log n) in best, average, and worst case, with O(1) extra space.

[MUST REMEMBER] Must Remember Rules & Formulas

- Stable algorithms preserve relative order of equal keys: Bubble, Insertion, Merge, Counting, Radix.
- Unstable algorithms: Selection, Quick, Heap.

[COMMON EXAM TRAP / CORRECTION] Common Confusions & Exam Traps

- Trap: "Quick Sort is always O(n log n)." -> WRONG! Worst case is O(n^2).
- Trap: "Selection sort best case is O(n)." -> WRONG! Selection sort is ALWAYS O(n^2) because it must find the minimum.

Unit 11: Critical Audit & Corrections of Original Class Slides

Class Reference: Quality Assurance & Error Verification

11.1 Identified Errors in Original Notes and Official Corrections

Examiner alert: Preparing from class slides without verifying technical facts can cost precious marks in CUET PG. The following errors in the course notes have been systematically verified and corrected:

- **Correction 1 (Class 01 Slide 6 / Note Page 6 & 12 - Graph Degree Error):** Original Slide Text: "Node (4) in below figure has indegree 3, outdegree 6."
  VERIFIED CORRECTION: This is mathematically incorrect. Node 4 has 3 incoming edges (from 1, 2, 3) so In-degree = 3. Node 4 has 3 outgoing edges (to 5, 6, 7) so Out-degree = 3. The TOTAL degree is In-degree + Out-degree = 3 + 3 = 6. The slide confused Out-degree with Total Degree!
- **Correction 2 (Class 04 Slide 5 - Linear Queue Front/Rear Conditions):** Original Note Text: "Insertion Rule: REAR = REAR + 1, Deletion Rule: FRONT = FRONT + 1".
  VERIFIED CORRECTION: This rule only applies when queue is neither full nor empty. For the very first insertion, BOTH FRONT and REAR must be set to 0. When the last element is deleted, FRONT and REAR must be reset to -1 (empty state).
- **Correction 3 (Class 08 Slide 2 - B-Tree Minimum Children Rule):** Original Note Text: "Every internal node must have at least ceil(m/2) children."
  VERIFIED CORRECTION: Important exception omitted! The ROOT node is an internal node, but the root only needs at least 2 children (not ceil(m/2)). The ceil(m/2) rule applies strictly to NON-ROOT internal nodes.
- **Correction 4 (Class 10 Slide 15 - Handshaking Lemma Application):** Important Rule Clarification: In any graph, sum of degrees = 2E. A question often asks: "Can a graph have 5 vertices of degree 3?" Answer: NO, because 5 * 3 = 15 (odd sum), but sum of degrees MUST be even (2E). The number of odd-degree vertices must ALWAYS be even!
- **Correction 5 (Class 11 Slide 12 - Selection Sort Stability):** Clarification: Selection sort is inherently UNSTABLE (e.g., sorting [4a, 4b, 2] places 2 first and swaps with 4a, resulting in [2, 4b, 4a], swapping the relative order of 4a and 4b).

Exam-Focused Review: Unit 11

[MOST IMPORTANT FOR CUET PG] Most Important for CUET PG

- Always verify graph degree sum: odd sum is impossible.
- Root node of B-tree has at least 2 children, whereas internal nodes have at least ceil(m/2) children.

[MUST REMEMBER] Must Remember Rules & Formulas

- Handshaking theorem is a guaranteed scoring question in CUET PG.

[COMMON EXAM TRAP / CORRECTION] Common Confusions & Exam Traps

- Never trust slide typos blindly; rely on formal standard definitions.

Unit 12: Master Comparison Tables

Class Reference: Cross-Topic Synthesis

12.1 Core Architectural Comparisons

Feature	Array	Linked List
Memory Allocation	Static (fixed compile-time size)	Dynamic (allocated at runtime as needed)
Memory Contiguity	Contiguous memory locations	Non-contiguous memory connected by pointers
Element Access	O(1) Random Access using index	O(n) Sequential traversal required
Insertion/Deletion	O(n) due to element shifting	O(1) if pointer to position is known
Memory Overhead	Zero extra memory overhead	Requires extra space for pointer(s) per node

Feature	Stack	Queue
Access Principle	LIFO (Last In First Out)	FIFO (First In First Out)
Active Ends	Single end (TOP) for both insert and delete	Two ends: REAR for insert, FRONT for delete
Core Operations	PUSH (insert), POP (delete), PEEK	ENQUEUE (insert), DEQUEUE (delete)
Primary Applications	Function calls, Undo/Redo, Expression parsing	CPU Scheduling, Print spooling, Breadth-First Search

Exam-Focused Review: Unit 12

[MOST IMPORTANT FOR CUET PG] Most Important for CUET PG

- Review these matrices right before entering the exam hall.

[MUST REMEMBER] Must Remember Rules & Formulas

- Array is best for frequent access; Linked List is best for frequent insertion/deletion.

[COMMON EXAM TRAP / CORRECTION] Common Confusions & Exam Traps

- Confusing LIFO and FIFO.

Unit 13: CUET PG Rapid Revision Cheat Sheet

Class Reference: Final Exam Preparation & Formula Digest

13.1 High-Yield Formula Summary for CUET PG

[MOST IMPORTANT FOR CUET PG] Master Formula Digest (Revise Morning of Exam!)

1. 1D Array Address: Loc(A[i]) = B + W * (i - LB)
2. 2D Array RMO: Loc(A[i][j]) = B + W * [(i - LB1) * C + (j - LB2)]
3. 2D Array CMO: Loc(A[i][j]) = B + W * [(j - LB2) * R + (i - LB1)]
4. Circular Queue: REAR = (REAR + 1) % N | Full: (REAR + 1) % N == FRONT
5. Tree Edges: For any tree with n vertices, |E| = n - 1
6. Binary Tree Leaves: n0 = n2 + 1 (Leaves = Degree-2 nodes + 1)
7. Max nodes in Binary Tree of height h: 2^(h+1) - 1
8. Distinct Binary Trees with n nodes: Catalan Cn = (1 / (n + 1)) * (2n choose n)
9. AVL Tree Minimum Nodes: N(h) = N(h - 1) + N(h - 2) + 1 [N(0)=1, N(1)=2, N(2)=4, N(3)=7]
10. Handshaking Lemma: Sum deg(v) = 2 * |E|
11. Complete Graph Kn Edges: n * (n - 1) / 2
12. Planar Graph Euler Formula: V - E + F = 2

Exam-Focused Review: Unit 13

[MOST IMPORTANT FOR CUET PG] Most Important for CUET PG

- Remember n0 = n2 + 1 and Catalan numbers.

[MUST REMEMBER] Must Remember Rules & Formulas

- Heap Sort and Merge Sort are always O(n log n).

[COMMON EXAM TRAP / CORRECTION] Common Confusions & Exam Traps

- Never forget modulo wrap in circular queues.